

RTL8762E SDK User Guide

V1.3

2022/12/23

Realtek Confidential

修订历史

日期	版本	修改	作者	审阅
2022/03/14	V1.0	Keil Version	Lory	
2022/06/02	V1.1	修改第 6 节、第 8 节和 9.1 节 修改图 2-8, 图 10-1, 图 10-2	Grace	
2022/10/19	V1.2	增加 4.5 启动流程	Lory	
2022/12/23	V1.3	修正 7.3 中断优先级	Lory	

Realtek Confidential

目 录

1 概述	1
2 用户指南	2
2.1 开发环境	2
2.1.1 Keil.....	2
2.1.2 J-Link.....	2
2.1.3 SDK	3
2.1.4 EVB Kit	3
2.2 平台设置	3
2.3 EVB 安装	3
2.4 IDE 安装.....	4
2.4.1 Keil 工程.....	4
2.4.2 Keil 设置.....	4
3 硬件架构	8
4 软件架构	9
4.1 系统架构	9
4.2 操作系统	10
4.3 OS 接口	10
4.4 任务和优先级	11
4.4.1 任务	11
4.4.2 优先级	12
4.5 启动流程	13
5 应用程序	16
5.1 SDK 目录	16
5.2 示例工程	17
5.3 流程图	18
5.4 消息和事件处理流程	20
5.5 IO 消息	20
5.5.1 消息格式	20

5.5.2 消息类型定义.....	20
5.5.3 消息子类型定义.....	21
5.5.4 用户自定义消息.....	21
5.6 Pin 设置.....	21
5.7 DLPS 设置	22
6 存储	23
6.1 存储映射	23
6.2 ROM.....	23
6.3 RAM.....	23
6.3.1 Data Ram	24
6.3.2 Buffer Ram.....	24
6.4 Cache	24
6.5 Flash.....	24
6.5.1 Flash APIs.....	25
6.5.2 FTL	25
6.6 eFuse.....	25
7 中断	26
7.1 Nested Vectored Interrupt Controller (NVIC).....	26
7.2 中断向量表	26
7.3 中断优先级	30
8 电量管理.....	31
9 烧录	32
9.1 Images 相关.....	32
9.2 Image 处理工具	32
9.2.1 fromelf	33
9.2.2 prepend_header.....	33
9.2.3 md5	34
9.3 烧录方式	34
10 调试.....	35
10.1 Log 机制.....	35

10.1.1 Debug analyzer	35
10.1.2 Log 打印基本接口	36
10.1.3 Log 打印封装接口	37
10.1.4 辅助打印接口.....	38
10.1.5 Log 打印示例	38
10.1.6 Log 控制接口	39
10.1.7 DBG_DIRECT.....	41
10.2 SWD 调试	41
10.2.1 SWD 调试接口.....	42

Realtek Confidential

表目录

表 4-1 RTL8762E 任务描述	11
表 5-1 SDK 目录描述	16
表 5-2 SDK 公共文件说明	17
表 5-3 APP 初始化	19
表 7-1 中断向量表	26
表 7-2 Timer 4~5 ISR	27
表 7-3 Peripheral ISR	28
表 7-4 GPIO Group3 ISR	28
表 7-5 GPIO Group2 ISR	28
表 7-6 GPIO Group1 ISR	29
表 7-7 GPIO Group0 ISR	29
表 7-8 中断优先级	30
表 9-1 Images 可用烧录方式	34
表 10-1 调试等级	37

图目录

图 2-1 Keil.....	2
图 2-2 平台设置.....	3
图 2-3 Keil 工程.....	4
图 2-4 Keil Flash 算法	4
图 2-5 Keil 设置.....	5
图 2-6 Keil 调试设置.....	5
图 2-7 Flash 下载算法	6
图 2-8 编译和运行	6
图 3-1 RTL8762E 硬件架构	8
图 4-1 RTL8762E 软件架构	9
图 4-2 OSIF 架构	10
图 4-3 RTL8762E 任务	11
图 4-4 任务优先级.....	12
图 4-5 启动流程图.....	13
图 4-6 Flash Image 校验和执行	14
图 4-7 Dual Bank 启动流程图.....	15
图 5-1 SDK 目录.....	16
图 5-2 PXP 工程示例.....	17
图 5-3 APP 流程.....	18
图 5-4 IO 消息处理流程.....	19
图 6-1 Memory Map.....	23
图 6-4 External Flash Layout	24
图 10-1 Debug Analyzer 主页面.....	35
图 10-2 Debug Analyzer 设置页面.....	36
图 10-3 RTL8762E 调试接口	41

1 概述

本文旨在说明如何使用 RTL8762E 软件开发包（SDK）来开发蓝牙低功耗应用。

Realtek Confidential

2 用户指南

2.1 开发环境

以下是开发过程中需要的软件和硬件。

1. Keil MDK-ARM Essential（或更高）V5（或更新）版本
2. J-Link Software v6.32（或更新）版本
3. RTL8762E SDK
4. EVB Kit

2.1.1 Keil

SDK 中所有的应用能够通过 Keil Microcontroller Development Kit(MDK)编译以及使用。所以在进行软件开发之前，要先取得并安装 Keil。有关 Keil 的更多信息请访问 www.keil.com。

Realtek 使用的工具链版本信息如下。为避免产生 ROM 可执行程序和应用之间的兼容性问题，建议使用以下版本或者更新的版本。

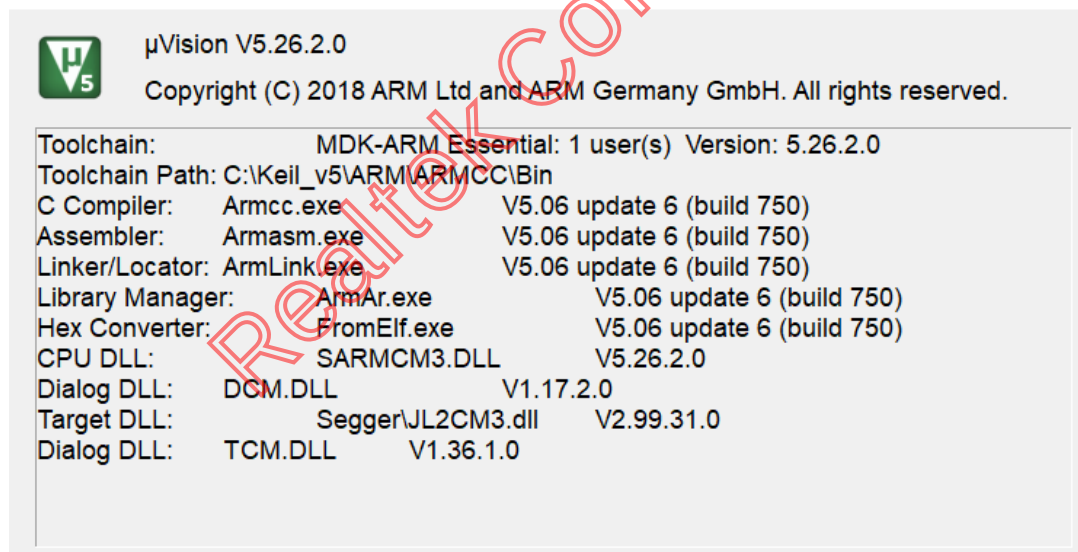


图 2-1 Keil

2.1.2 J-Link

相较于通过打印 log 来进行调试，基于 J-link 的调试方法是更为全面的。访问 www.segger.com 来下载相关的软件和文档，安装 SEGGER J-Link 软件。保证正确安装设备的驱动，才能搭配 Keil MDK 使用 J-Link 调试器。

2.1.3 SDK

SDK 包括示例工程，文档以及必要的工具。

2.1.4 EVB Kit

EVB 是针对 RTL8762E 的硬件评估，开发和应用调试设计的。

2.2 平台设置

图 2-2 展示了硬件部分和软件组件之间的关系。通过 Keil 开发的应用程序能够通过 J-Link 下载到 RTL8762E，并且通过 SWD 接口调试。

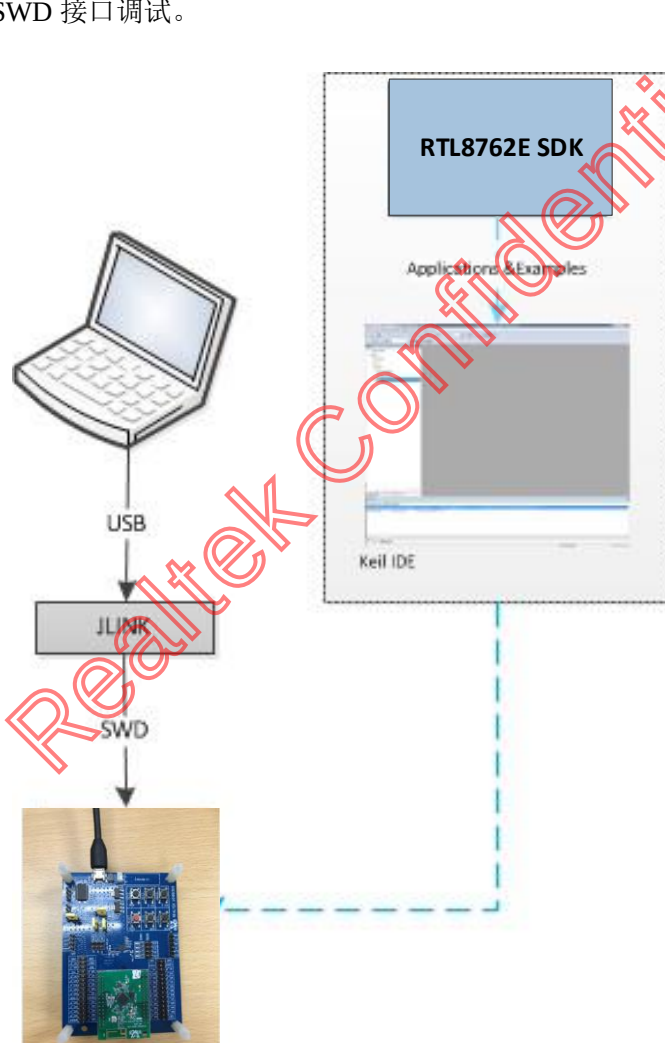


图 2-2 平台设置

2.3 EVB 安装

EVB 套件由一块母板和一块子板组成，详情请参考《RTL8762E Evaluation Board User Guide》。

2.4 IDE 安装

2.4.1 Keil 工程

一般来说不建议新建工程开发，而是直接打开现有的示例工程，然后在此基础上增加功能代码。

如果安装的是 Keil V5 或者更新的版本，以 PXP 工程为例（工程目录在 bee3_sdk_ xxxx \board\evb\pxp），打开方式如图 2-3 所示。

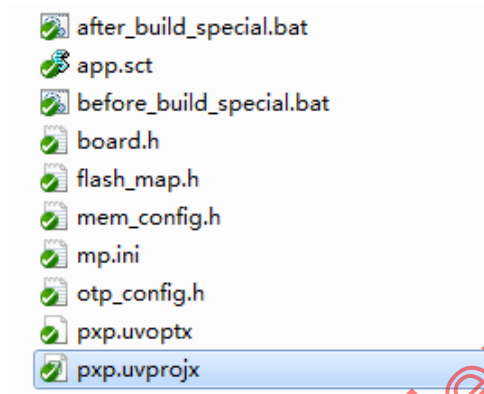


图 2-3 Keil 工程

2.4.2 Keil 设置

下面是对 Debugger 设定和 Keil 使用的 Flash 下载算法的描述。

1. 把 sdk\tool\flash 目录下的 Bee3_SPI_FLASH.FLM 和 RTL876x_LOG_TRACE.FLM 文件拷贝到 Keil 安装目录：Keil_installed_dir\ARMFlash\，如图 2-4 所示。

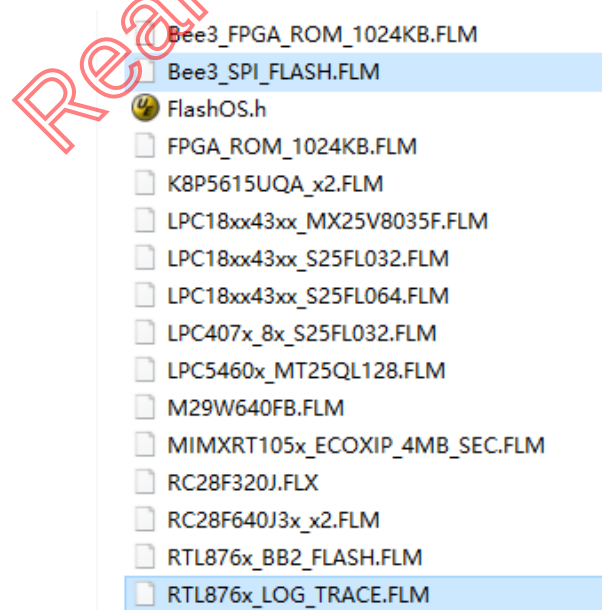


图 2-4 Keil Flash 算法

2. 点击工具栏上的，或者在菜单中进入 Project → Options，点击 Debug 页面，选择 J-LINK/J-TRACE Cortex，然后点击“Settings”。

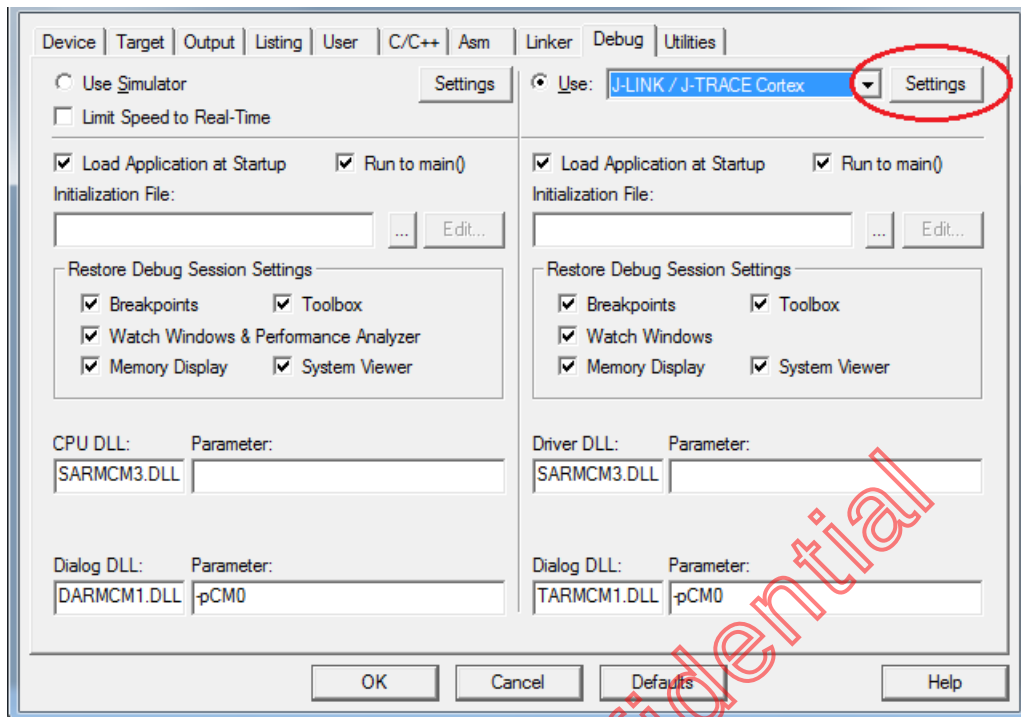


图 2-5 Keil 设置

3. J-Link Port 选择 SW。如果硬件连接是正确的，CPU 就能在 SW Device 列表中识别出来。

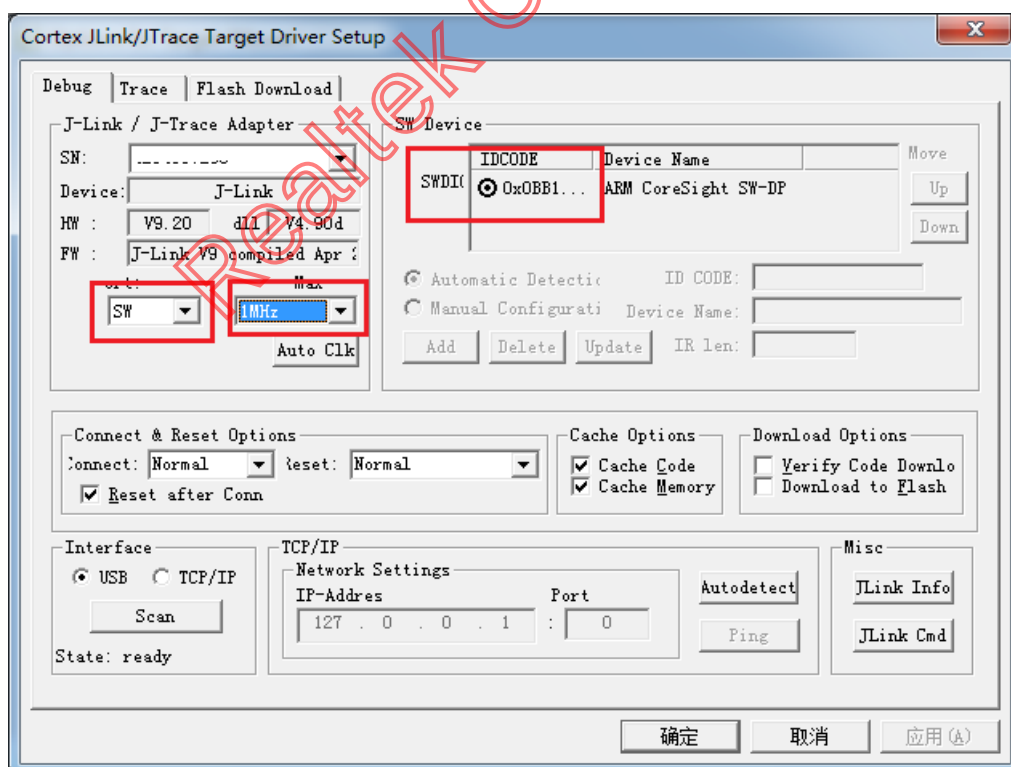


图 2-6 Keil 调试设置

- 选择“Flash Download”页面，如果已经有下载算法就先删除，点击“Add”添加 Bee3_SPI_FLASH 和 RTL876x_LOG_TRACE。修改“RAM for Algorithm”的起始地址为 0x00200000，大小为 0x4000。

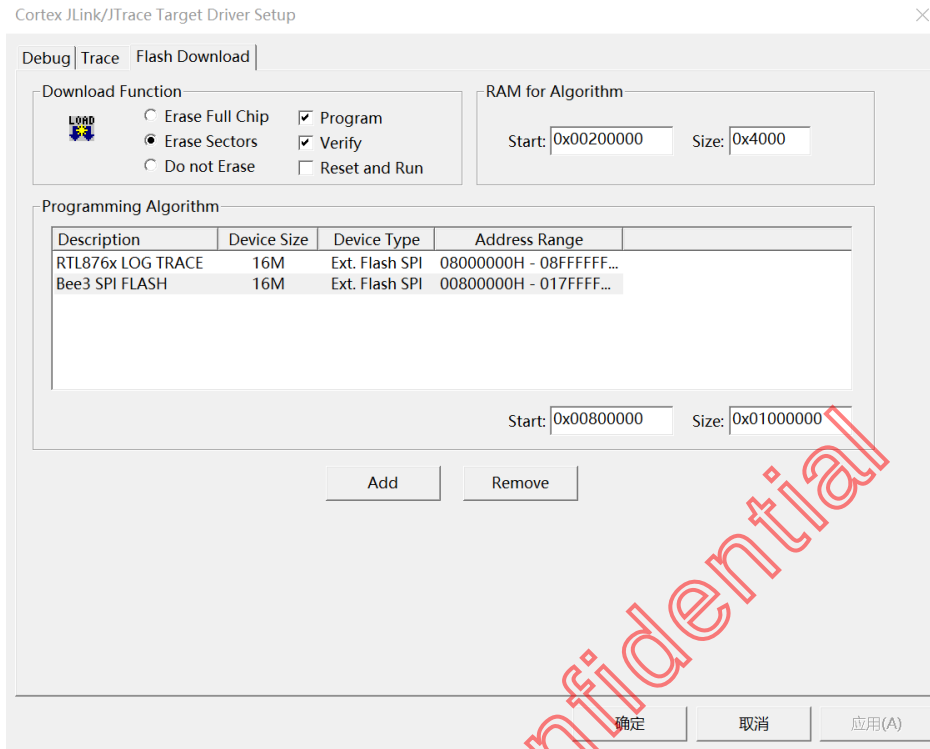


图 2-7 Flash 下载算法

- 点击图标  编译工程，如果有任何编译或者链接的错误，在解决完后就可以把应用程序下载到 RTL8762E 来做进一步的验证和调试。
- 如果 Flash 算法有成功的安装，就可以通过点击下载图标  来进行下载。在下载过程中，预期是会产生任何出错的信息。当应用程序被成功下载到 RTL8762E 后，重启 RTL8762E 来运行程序。可以通过 Debug Analyzer 工具上的日志信息来检查应用程序是否按照预期运行。

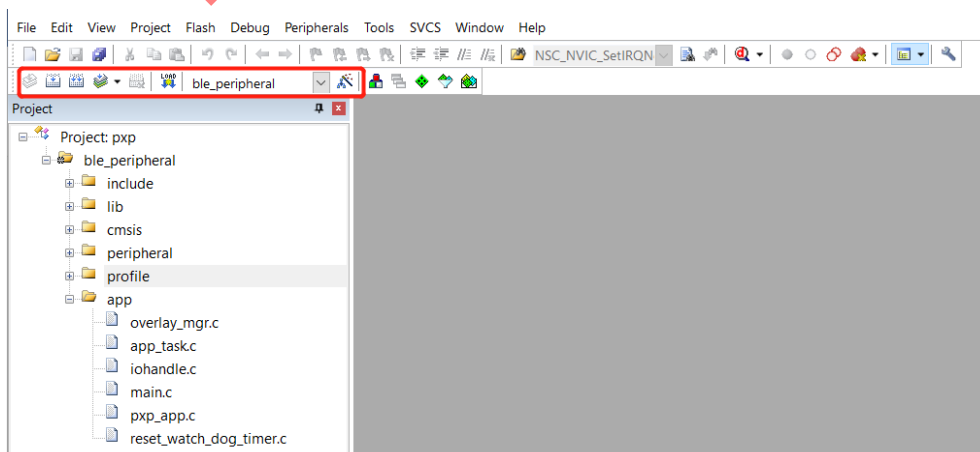


图 2-8 编译和运行

7. 通过点击调试图标, 利用 J-Link 来调试和追踪应用程序。

注意: 如果 DLPS 模式是打开的, 那么一旦系统进入 DLPS 模式, 调试就不能继续进行。所以想要在开发早期利用 Keil 来调试程序时, 要先关闭 DLPS 模式。

DLPS 打开和关闭方式:

- 打开: `lps_mode_set (PLATFORM_DLPS_PFM)`
- 关闭: `lps_mode_set (PLATFORM_ACTIVE)`

Realtek Confidential

3 硬件架构

RTL8762E 的硬件模块如图 3-1 所示，由以下部分组成。

- 丰富的外设
- 灵活的 RAM 配置
- 电源管理单元
- 时钟管理单元
- 蓝牙模块

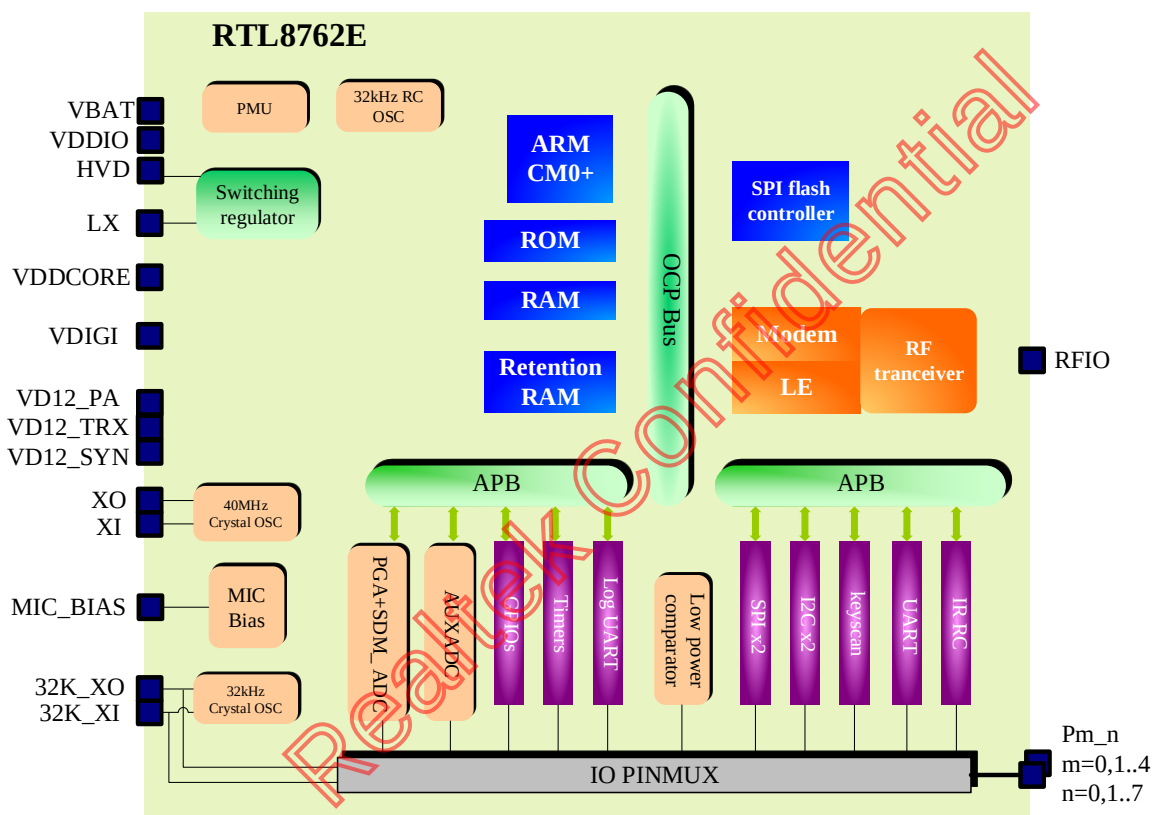


图 3-1 RTL8762E 硬件架构

4 软件架构

4.1 系统架构

如图 4-1 所示，软件架构包括几个主要的部分。

- **GAP:** 应用程序和 BLE 协议栈交互的抽象层，详情请参考《GAP Interfaces User Manual》
- **Platform:** 包括 OTA, flash, FTL 等
- **IO Drivers:** 提供应用层访问 RTL8762E 外设的接口，无需直接访问寄存器
- **OSIF:** 实时操作系统的抽象层

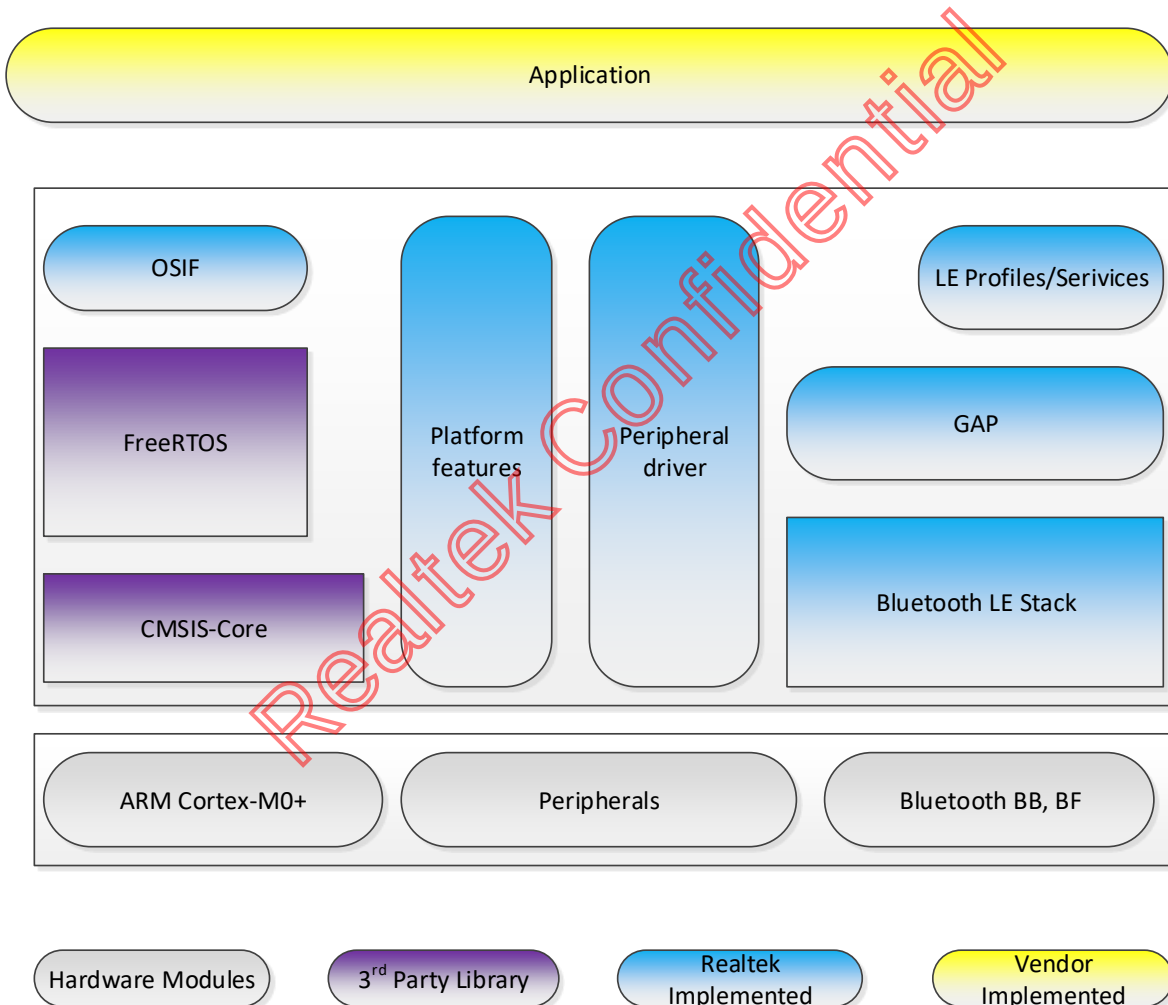


图 4-1 RTL8762E 软件架构

4.2 操作系统

RTL8762E 的操作系统是 FreeRTOS V10.2，该系统运行在 ROM 上，包括以下几个模块。

1. 任务管理
2. 队列管理
3. 中断管理
4. 资源管理
5. 内存管理
6. 时间管理

4.3 OS 接口

如图 4-2 所示，OSIF 层通过封装特定的 RTOS 接口来提供一层统一的接口。开发者可以基于 OSIF 层提供自定义的 RTOS 实现，而不需要在上层的软件做任何修改。所以，强烈推荐在软件开发中使用 OSIF 接口，而不是直接访问特定 RTOS 的接口。

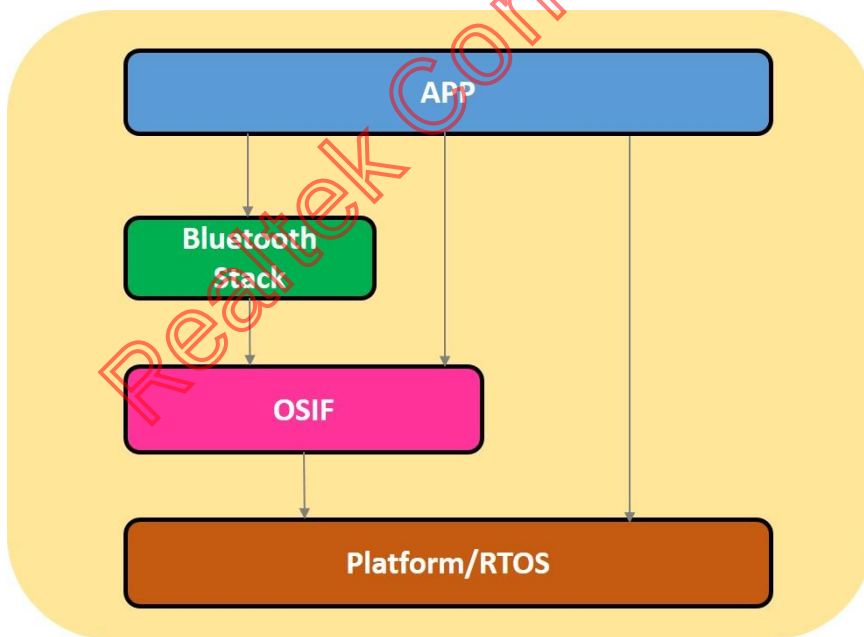


图 4-2 OSIF 架构

4.4 任务和优先级

4.4.1 任务

如图 4-3 所示，应用程序创建了 5 个任务。

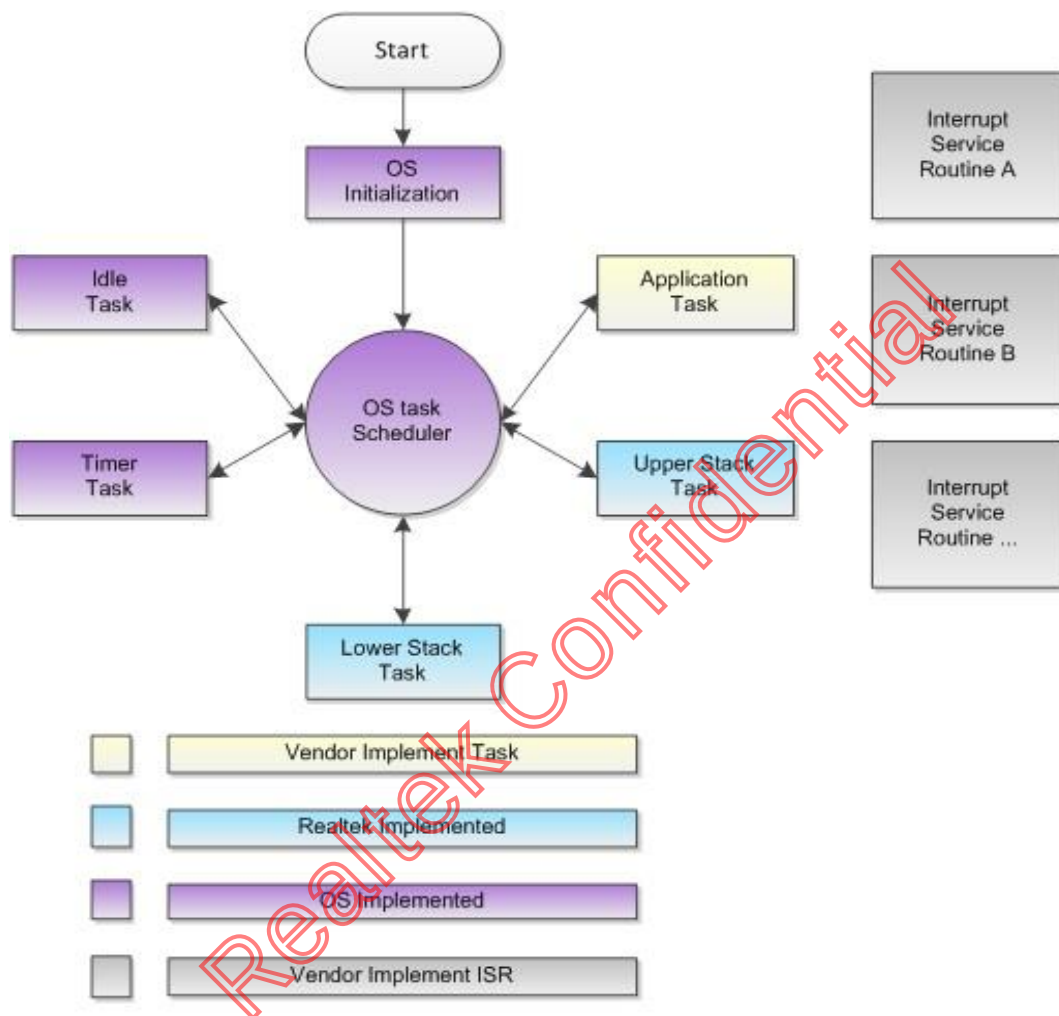


图 4-3 RTL8762E 任务

表 4-1 RTL8762E 任务描述

Task	Description	Priority
Timer	FreeRTOS 实现的软件定时器	6
Lower stack	蓝牙 HCI 层以下的协议栈实现	6
Upper stack	蓝牙 HCI 层以上的协议栈实现	5
App	应用层功能的实现	1~4
Idle	后台空闲任务，包括低功耗的处理	0

说明:

1. 可以创建多个 APP 任务，同时内存资源也会被相应地分配。
2. Idle 任务和 Timer 任务是 FreeRTOS 创建的。
3. 任务是配置成根据优先级抢占式的。
4. 硬件中断服务例程（ISR）是由 Vendor 来实现的。

4.4.2 优先级

任务分 7 种优先级：Priority_{SW Timer} = Priority_{lower stack} > Priority_{upper stack} > Priority_{app} > Priority_{idle}。

其中 APP 优先级可从 1~4 中选择。

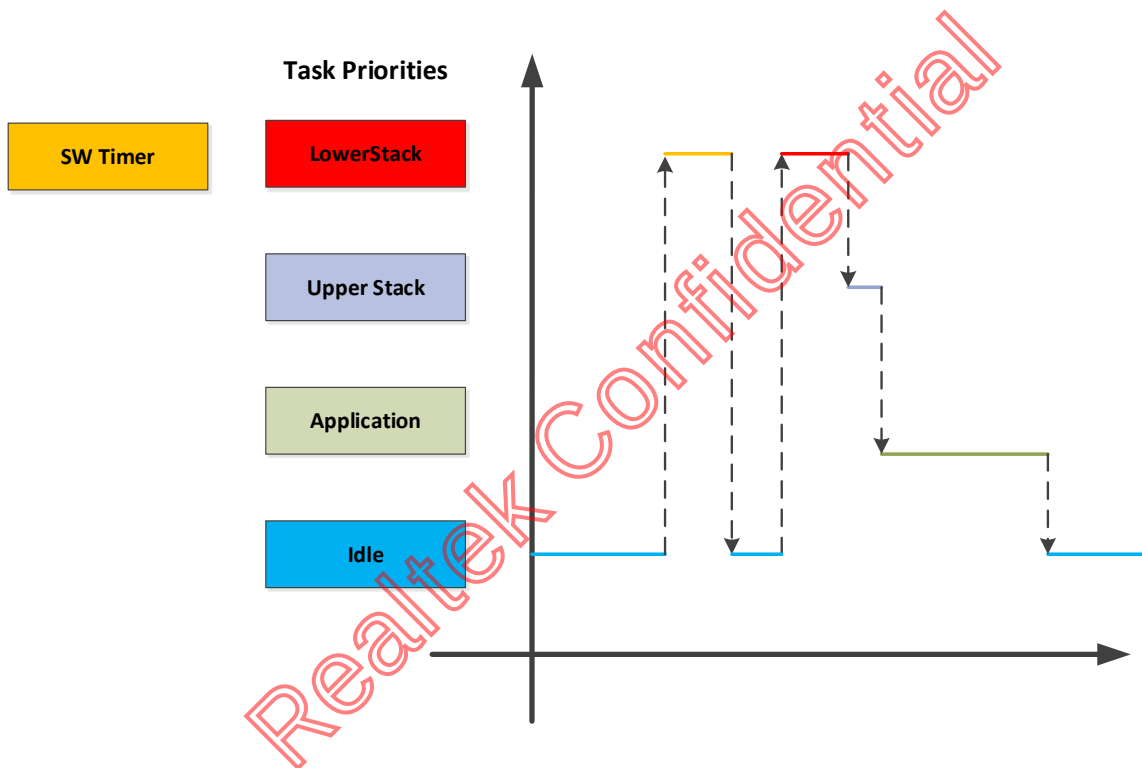


图 4-4 任务优先级

4.5 启动流程

系统上电或者重启，会执行启动流程，启动流程包括。

- eFuse 校验
- 根据 efuse 进行安全控制
- Flash image 校验和执行

启动流程如图 4-5 所示。

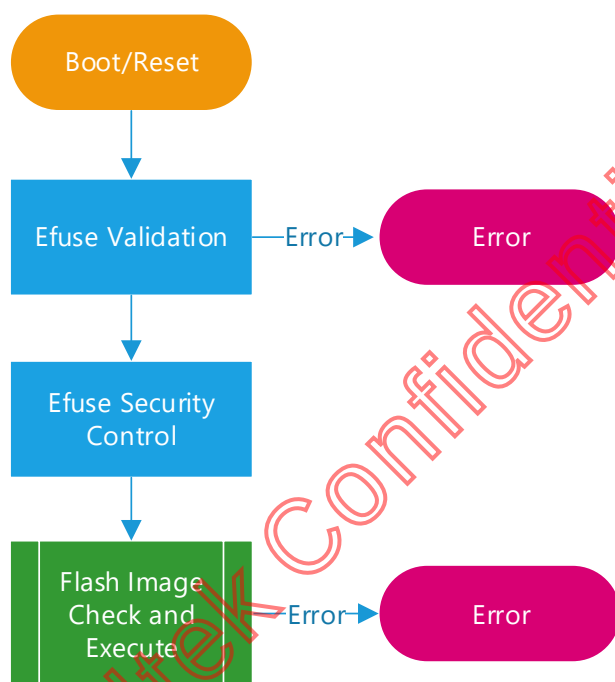


图 4-5 启动流程图

在启动流程图 4-5 中，“Flash Image Check and Execute”表示对各种 Flash Image 检查和执行，具体流程包括如下。

- 检查 FT Data
- 检查 OEM Data（Flash layout 包含在此文件中）
- 检查是否存在有效的待升级文件
- 检查 OTA Bank Header 文件（参见 6.5 章节）
- 检查 OTA Bank 中每一个 Flash image
- 解密 image
- 执行 image

图 4-6 是以上流程的流程图。Image 检查方式是 SHA256 完整性校验。执行 OTA bank 内 Flash image 的顺序：Secure Boot Loader Image、Patch Image、App Image 和 Upper Stack Image。

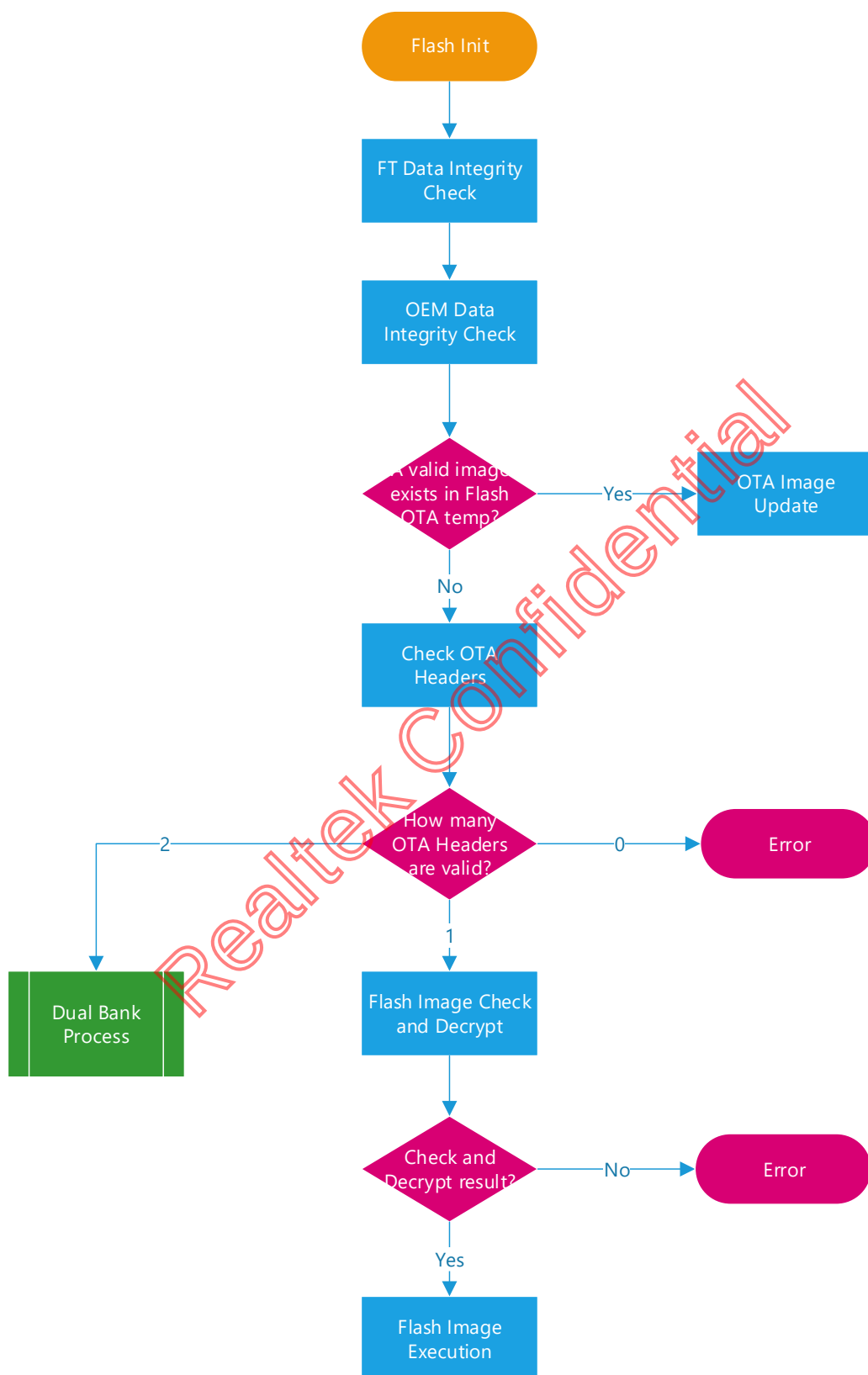


图 4-6 Flash Image 校验和执行

图 4-6 中的“Dual Bank Process”是针对切换 Bank 模式升级 Image 的处理，在一个旧版本上升级一个新版本，Bank0 和 Bank1 中都会存在 images，启动流程会优先校验版本号高的 OTA Header，如果校验和解密成功，则执行该 Bank 内 images；如果校验不成功，会继续校验版本号低的 OTA Header。

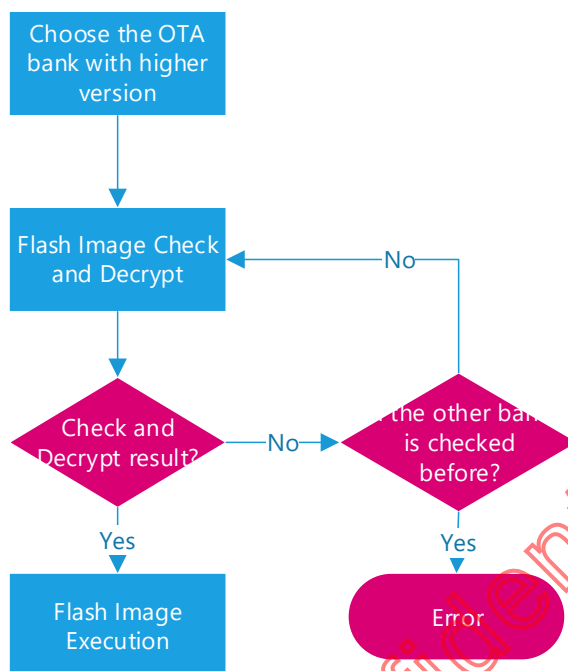


图 4-7 Dual Bank 启动流程图

5 应用程序

5.1 SDK 目录

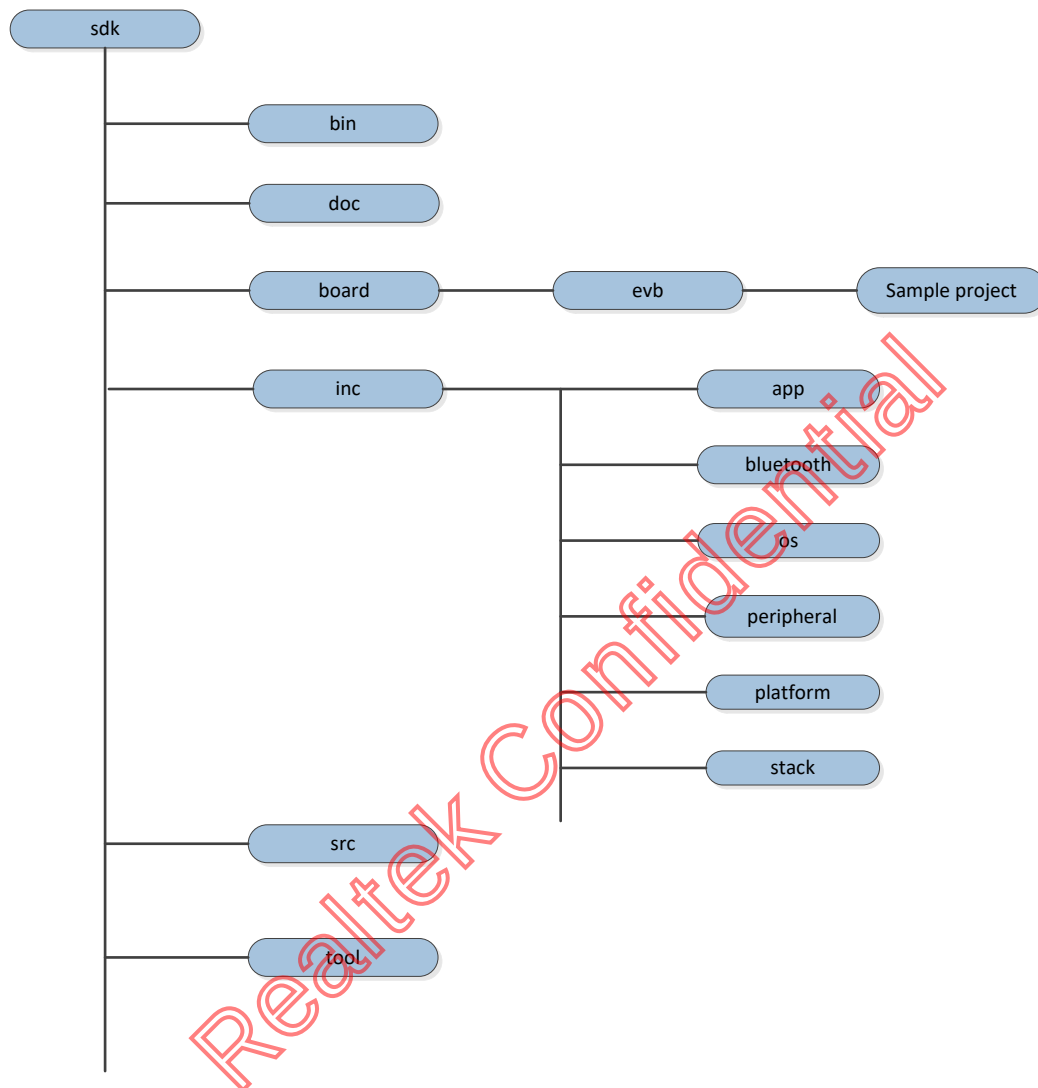


图 5-1 SDK 目录

表 5-1 SDK 目录描述

Directory	Description
board	配置好可直接使用的 Keil 示例工程
doc	文档
inc	提供接口定义的头文件
bin	应用程序链接的二进制文件
src	应用程序的源码文件
tool	工具集

5.2 示例工程

为了帮助创建应用程序，SDK 中已经创建好了许多示例工程，例如 PXP 以及一些 BLE 相关的示例工程。通过学习示例工程，开发者可以很容易地熟悉 SDK。所有的示例工程已经配置好，并且根据 RTL8762E SOC 划分好内存。

下面以 PXP 为例，展示如何通过示例工程来开发客制化应用程序。下面的图摘自当前 SDK，实际可能会随着 SDK 的更新而变化。

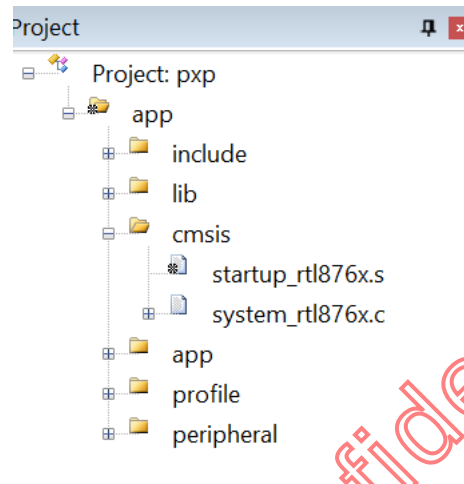


图 5-2 PXP 工程示例

PXP 工程中的文件分为以下几个类别。

1. **Include 目录：**主要用来导出 UUID
(注：UUID 是任何在 RTL8762E 运行的应用所必须的，应用开发者不能修改此目录)
2. **Lib 目录：**应用程序使用的所有二进制文件
3. **CMSIS 目录：**存放启动代码
4. **APP 目录：**PXP 应用的实现
5. **Profile 目录：**应用使用的 BLE profiles 或者服务
6. **Peripheral 目录：**应用使用的所有外设驱动和模块代码

应用中的一些公共文件说明如下。

表 5-2 SDK 公共文件说明

File name	Description
rom_uuid.h	UUID 头文件是 SDK 用来识别 ROM 的，无需更改
ROM.lib	ROM 符号库，应用程序链接 ROM 中的符号
gap_utils.lib	实现 BLE 功能的 Gap 库
startup_rtl876x.s	RTL8762E 应用程序启动的汇编文件

File name	Description
system_rtl876x.c	RTL8762E 应用程序启动的 C 文件
board.h	配置引脚和 DLPS 的头文件
flash_map.h	Flash layout 文件，此文件由 FlashMapGenerateTool 工具生成
mem_config.h	Memory 配置相关的文件

示例工程可能会随着 SDK 一起更新。为了更好地使用最新的示例代码，建议把新增的用户代码组织起来模块化。

每个示例工程的详细信息请参考对应示例工程的使用手册。

5.3 流程图

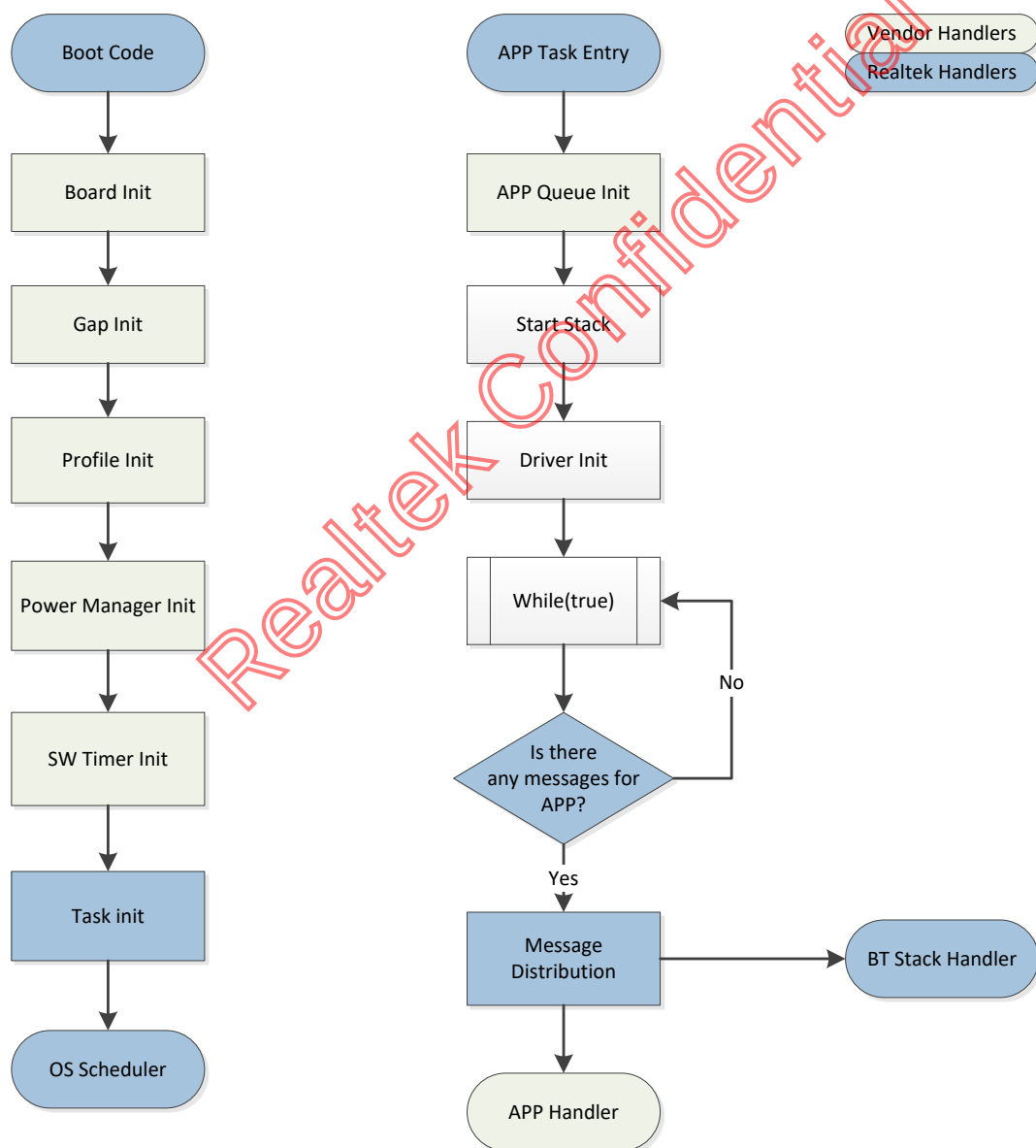


图 5-3 APP 流程

表 5-3 APP 初始化

Action	Description
board init	PINMUX 和 PAD 初始化设定
gap init	GAP 相关参数的初始化
profile init	BLE Profiles 的初始化
power manager init	电源管理相关的初始化
SW timer init	软件定时器的初始化
app queue init	APP Queue 的初始化
driver init	外设的初始化

系统是在 main() 函数中初始化，包括：Board、Peripherals、BT Stack、Profile、Power 和 Task 等。

BT Stack，Profiles 和外设驱动是在应用层任务中初始化，并且实现了 IO 消息机制。所有的功能都被封装成 IO 事件，这些事件是在相关的消息处理器中处理。

BT Stack 消息也是被封装成 BT IO 事件，跟外设事件处理的方式一样。

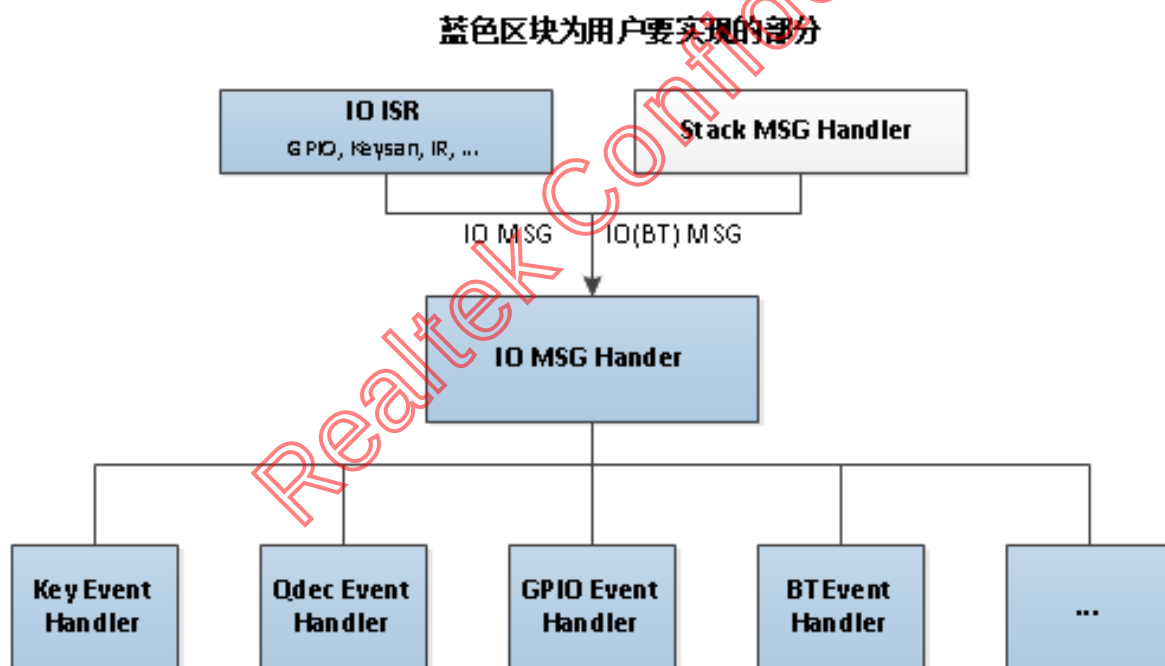


图 5-4 IO 消息处理流程

5.4 消息和事件处理流程

1. 原始消息的来源有通用 Peripherals 的 ISR 和 BT Stack，处理流程如下。
 - 1) 来自通用 Peripherals 的 MSG 会被 MSG 分发器转发给 IO MSG Handler 处理。
 - 2) 来自蓝牙协议栈的 MSG 会被 MSG 分发器转发给 BT 状态机，BT 状态机处理完 MSG 后，会封装成 BT IOMSG 发出，MSG 分发器收到 BT IO MSG 后再转发给 IO MSG Handler 处理。
 - 3) IO MSG Handler 收到 MSG 后再判断 Event 类别，进而调用对应的 Event Handler 处理。
2. 用户程序负责以下事项。
 - 1) 实现 Peripheral ISR，在 ISR 中完成初步处理，如果需要进一步处理则封装 MSG 发给 APP。
 - 2) 维护 IO MSG Handler，以便接收和处理用户定义的 ISR MSG。
 - 3) 实现应用层相关的 Event Handler。
3. 底层通知上层是通过 MSG 和 Event 机制，而上层可以直接调用底层的 APIs。

5.5 IO 消息

5.5.1 消息格式

```

1.  typedef struct
2.  {
3.      uint16_t type;
4.      uint16_t subtype;
5.      union
6.      {
7.          uint32_t param;
8.          void *buf;
9.      }u;
10. }T_IO_MSG;
  
```

5.5.2 消息类型定义

```

1.  typedef enum
2.  {
3.      IO_MSG_TYPE_BT_STATUS,
  
```

```

4.    IO_MSG_TYPE_KEYSCAN,
5.    IO_MSG_TYPE_QDECODE,
6.    IO_MSG_TYPE_UART,
7.    IO_MSG_TYPE_KEYPAD,
8.    IO_MSG_TYPE_IR,
9.    IO_MSG_TYPE_GDMA,
10.   IO_MSG_TYPE_ADC,
11.   ...
12. }T_IO_MSG_TYPE;

```

5.5.3 消息子类型定义

以 T_IO_MSG_UART 为例，定义 UART 的子类型。

```

1.  typedef enum
2.  {
3.      IO_MSG_UART_RX                = 1,
4.      IO_MSG_UART_RX_TIMEOUT        = 2,
5.      IO_MSG_UART_RX_OVERFLOW       = 3,
6.      IO_MSG_UART_RX_TIMEOUT_OVERFLOW = 4,
7.      IO_MSG_UART_RX_EMPTY          = 5,
8.  }T_IO_MSG_UART;

```

5.5.4 用户自定义消息

APP 开发者可以根据需求扩充定义 Message Type 以及自定义 Subtype Message。

5.6 Pin 设置

引脚的定义在 board.h 中定义。

```

1. #define KEY_0  P4_0
2. #define BEEP  P4_1
3. #define LED_0  P2_1
4. #define LED_1  P2_4

```

5.7 DLPS 设置

DLPS 的配置是在 board.h 中，如下所示。

1. DLPS 功能开关。

```
1. #define DLPS_EN 1
```

2. 用户自定义的 DLPS CB，在进入 DLPS 或者退出 DLPS 时调用这两个 CB。

```
1. #define USE_USER_DEFINE_DLPS_EXIT_CB 1
2. #define USE_USER_DEFINE_DLPS_ENTER_CB 1
```

3. 外设功能开关，需要使用的外设必须对应打开，这样在进入 DLPS 时才会保存外设的寄存器设定，退出 DLPS 的时候恢复外设的寄存器设定，没有使用的外设请关闭。（0 表示关闭，1 表示打开）

```
1. #define USE_I2C0_DLPS 0
2. #define USE_I2C1_DLPS 0
3. #define USE_TIM_DLPS 0
4. #define USE_QDECODER_DLPS 0
```

6 存储

6.1 存储映射

RTL8762E 内存包括 ROM, RAM, external SPI Flash 和 eFuse。Flash Cache 可以配置成 Cache 来使用, 也可以配置成 Data RAM 来使用, 如图 6-1 所示, 详情请参考《RTL8762E Memory User Guide》。

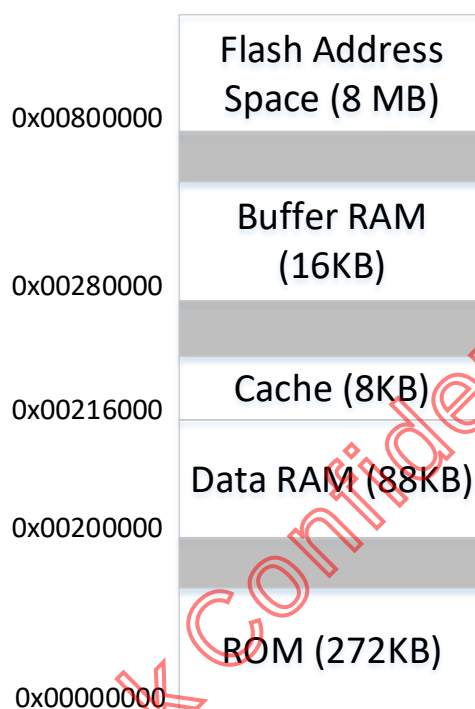


图 6-1 Memory Map

6.2 ROM

ROM 代码位于地址[0x0, 0x44000), 其中包括 Boot loader、RTOS、BT Stack、Flash Driver 以及其它平台模块。

6.3 RAM

RTL8762E 有两块 RAM。

- Data RAM: 位置在[0x00200000, 0x00216000)
- Buffer RAM: 位置在[0x00280000, 0x00282000)

Cache 可以被配置成 Data RAM, 地址: [0x00216000, 0x00218000)

6.3.1 Data Ram

Data RAM 主要用于 ROM 静态数据存储、主栈、Patch/Upperstack/APP 静态数据存储和代码执行以及系统动态内存的管理，更多详细信息请参考《RTL8762E Memory User Guide》

6.3.2 Buffer Ram

Buffer RAM 主要用于 ROM 的静态数据存储和蓝牙协议栈的动态内存申请，更多详细信息请参考《RTL8762E Memory User Guide》

6.4 Cache

RTL8762E 有一块 8K 字节的 cache, 和 SPIC(SPI Flash Controller)一起工作以加快 SPI Flash 读操作。Cache 也可以当成 Data RAM 使用，如果 Cache 被配置成 Data RAM，可以用作数据存储也可以用作代码执行，地址范围为[0x00216000, 0x00218000)，紧跟在 Data RAM 后面。

6.5 Flash

RTL8762E 支持最大 8M 字节大小的 external SPI Flash。在 SDK 中，external Flash 默认分成 7 份，如图 6-2 所示。

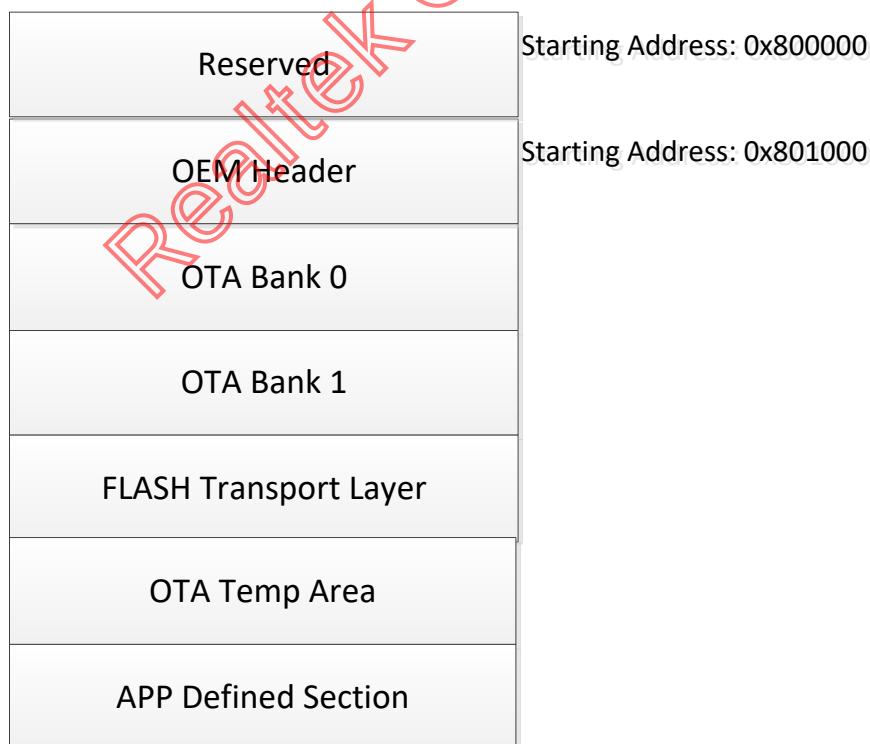


图 6-2 External Flash Layout

6.5.1 Flash APIs

Flash 操作的 APIs 如下所列，详细请查阅 Bee3-SDK.chm 和《RTL8762E Flash User Guide》

1. **bool** flash_read_locked(uint32_t start_addr, uint32_t data_len, uint8_t *data);
2. **bool** flash_auto_read_locked(uint32_t addr, uint32_t *data);
3. **bool** flash_split_read_locked(uint32_t start_addr, uint32_t data_len, uint8_t *data, uint32_t *counter);
4. **bool** flash_write_locked(uint32_t start_addr, uint32_t data_len, uint8_t *data);
5. **bool** flash_auto_write_locked(uint32_t start_addr, uint32_t data);
6. **bool** flash_auto_write_buffer_locked(uint32_t start_addr, uint32_t *data, uint32_t len);
7. **bool** flash_erase_locked(T_ERASE_TYPE type, uint32_t addr);

6.5.2 FTL

FTL (flash transport layer) 是 BT Stack 和用户程序读写 flash 数据的抽象层。FTL 更多详细信息请参考《RTL8762E Memory User Guide》

1. BT 存储空间

- 1) 用于存储 BT 信息，例如设备地址，link key 等
- 2) 详细请参考《RTL8762E BLE Stack User Manual》

2. APP 存储空间

- 1) 用于存储 APP 的信息
- 2) 可用以下 APIs 读/写 APP 存储数据，详细请参考 Bee3-SDK.chm

1. uint32_t ftl_save(void * p_data, uint16_t offset, uint16_t size)
2. uint32_t ftl_load(void * p_data, uint16_t offset, uint16_t size)

6.6 eFuse

eFuse 是一次编程的存储体，用来存储重要且固定的信息，例如：UUID，security key 以及其它只会配置一次的信息。eFuse 中每个 bit 不能从 0 改成 1，并且也没有擦除的操作，所以更新 eFuse 要十分小心。Realtek 提供 MP Tool 更新 eFuse。

7 中断

7.1 Nested Vectored Interrupt Controller (NVIC)

NVIC 特性

- 16 个 Cortex-M0+ 异常，32 条可屏蔽的中断通道
- 8 个可编程的中断优先级
- 支持向量表的重映射
- 低延迟异常和中断处理
- 系统控制寄存器（System Control Registers）的实现

NVIC 和处理器内核接口紧密耦合，使得低延迟中断处理和高效处理晚抵达中断成为可能。所有的中断包括内核异常都由 NVIC 管理。

7.2 中断向量表

表 7-1 中断向量表

Exception Number	NVIC Number	Exception Type	Description
1		Reset	Reset
2		NMI	Nonmaskable interrupt. The WDG is linked to the NMI vector
3		Hard Fault	All fault conditions if the corresponding fault handler is not enabled
4~10		RSVD	
11		SVC	Supervisor Call
12~13		RSVD	
14		PendSV	Pendable Service Call
15		SYSTICK	System Tick Timer
16	[0]	System_ISR	System interrupt
17	[1]	WDG	Watch dog global interrupt
18	[2]	BTMAC_ISR	BT MAC interrupt
19	[3]	Timer3	Timer3 global interrupt
20	[4]	Timer2	Timer2 global interrupt

Exception Number	NVIC Number	Exception Type	Description
21	[5]	Platform	Platform interrupt
22	[6]	I2S_0 TX	I2S_0 TX interrupt
23	[7]	I2S_0 RX	I2S_0 RX interrupt
24*	[8]	Timer[4:5]	Timer4-5 interrupt (参考表 7-2)
25	[9]	GPIO4	GPIO interrupt, P0_4
26	[10]	GPIO5	GPIO interrupt, P0_5
27	[11]	RTK_UART1	RTK_UART1 interrupt
28	[12]	RTK_UART0	RTK_UART0 interrupt
29	[13]	RTC	Real time counter interrupt
30	[14]	SPI_0	SPI_0 interrupt
31	[15]	SPI_1	SPI_1 interrupt
32	[16]	I2C_0	I2C_0 interrupt
33	[17]	I2C_1	I2C_1 interrupt
34	[18]	ADC	ADC global interrupt
35*	[19]	Peripheral_ISR	Peripheral interrupt(参考表 7-3)
36	[20]	GDMA0_Channel0	RTK-DMA0 channel 0 global interrupt
37	[21]	GDMA0_Channel1	RTK-DMA0 channel 1 global interrupt
38	[22]	GDMA0_Channel2	RTK-DMA0 channel 2 global interrupt
39	[23]	GDMA0_Channel3	RTK-DMA0 channel 3 global interrupt
40	[24]	Enhanced_Timer0_Handler	Enhanced Timer0 interrupt
41	[25]	Enhanced_Timer1_Handler	Enhanced Timer1 interrupt
42*	[26]	GPIO_Group3	GPIO group3 interrupt(参考表 7-4)
43*	[27]	GPIO_Group2	GPIO group2 interrupt(参考表 7-5)
44	[28]	IR	IR module global interrupt
45*	[29]	GPIO_Group1	GPIO group1 interrupt(参考表 7-6)
46*	[30]	GPIO_Group0	GPIO group0 interrupt(参考表 7-7)
47	[31]	RSVD	

表 7-2 Timer 4~5 ISR

Exception Number	NVIC Number	Exception Type	Description
48	[8]	Timer4	Timer4 interrupt
49	[8]	Timer5	Timer5 interrupt

表 7-3 Peripheral ISR

Exception Number	NVIC Number	Exception Type	Description
50	[19]	SPIC0_Flash	SPIC0_Flash interrupt
51	[19]	qdecode	qdecode global interrupt
52	[19]	keyscan	keyscan global interrupt
53	[19]	2-wire SPI	2-wire/3-wire SPI interrupt
54	[19]	analog comparator	analog comparator interrupt
55	[19]	MailBox	MailBox interrupt
56	[19]	CAP Touch	CAP Touch interrupt
57	[19]	TRNG	TRNG interrupt

表 7-4 GPIO Group3 ISR

NVIC Number	Exception Type	Description
[26]	GPIO3	GPIO3 interrupt
[26]	GPIO7	GPIO7 interrupt
[26]	GPIO11	GPIO11 interrupt
[26]	GPIO15	GPIO15 interrupt
[26]	GPIO19	GPIO19 interrupt
[26]	GPIO23	GPIO23 interrupt
[26]	GPIO27	GPIO27 interrupt
[26]	GPIO31	GPIO31 interrupt

表 7-5 GPIO Group2 ISR

NVIC Number	Exception Type	Description
[27]	GPIO2	GPIO2 interrupt
[27]	GPIO6	GPIO6 interrupt
[27]	GPIO10	GPIO10 interrupt
[27]	GPIO14	GPIO14 interrupt
[27]	GPIO18	GPIO18 interrupt
[27]	GPIO22	GPIO22 interrupt
[27]	GPIO26	GPIO26 interrupt
[27]	GPIO30	GPIO30 interrupt

表 7-6 GPIO Group1 ISR

NVIC Number	Exception Type	Description
[29]	GPIO1	GPIO1 interrupt
[29]	GPIO9	GPIO9 interrupt
[29]	GPIO13	GPIO13 interrupt
[29]	GPIO17	GPIO17 interrupt
[29]	GPIO21	GPIO21 interrupt
[29]	GPIO25	GPIO25 interrupt
[29]	GPIO29	GPIO29 interrupt

表 7-7 GPIO Group0 ISR

NVIC Number	Exception Type	Description
[30]	GPIO0	GPIO0 interrupt
[30]	GPIO8	GPIO8 interrupt
[30]	GPIO12	GPIO12 interrupt
[30]	GPIO16	GPIO16 interrupt
[30]	GPIO20	GPIO20 interrupt
[30]	GPIO24	GPIO24 interrupt
[30]	GPIO28	GPIO28 interrupt

7.3 中断优先级

RTL8762E 中断支持 3 个固定最高优先级和 4 个可编程优先级，但为了和 RTL8762C，RTL8762D 兼容，都是按照 8 级中断优先级设定。优先级数字代表的优先级高低如下。

- RTL8762C/RTL8762D: $0 > 1 > 2 > 3 > 4 > 5 > 6 > 7$
- RTL8762E: $(0=1) > (2=3) > (4=5) > (6=7)$

表 7-8 中断优先级

Priority	Usage
-3	Reset Handler
-2	NMI
-1	Hard Fault
0/1	BT 相关的中断
2/3	通用的中断
4/5	
6/7	SysTick 和 PendSV

RTL8762E（ARMv6-M 架构）不支持动态调整 active 或者 enabled 中断的优先级。如果中断优先级要调整，需要先关闭中断，修改中断优先级，再重新使能中断。RTL8762C 和 RTL8762D（ARMv7-M 架构）与之不同，支持动态调整 active 或者 enabled 中断优先级。

8 电量管理

RTL8762E 会在满足特定的条件时进入低功耗模式，并且可以通过 PAD、BT 中断、RTC 中断和 OS software timer 到期唤醒，详细请参考文档《RTL8762E Deep Low Power State》。

Realtek Confidential

9 烧录

9.1 Images 相关

RTL8762E BLE 应用需要 6 个 images，说明如下。

1. Configuration file

记录 IC 硬件配置和蓝牙配置等信息，例如：配置 BT 地址，修改 Link 数目等，可通过 MP Tool 配置生成。

2. OTA Header File

定义 Flash Bank layout 的文件，由 MPPackTool 生成。

3. Secure Boot Loader Image

Realtek 发布，优化 Boot Flow。

无论 OTA 方式是否切 bank 的，都一定要烧录 Secure Boot Loader Image。

4. Patch Image

Realtek 发布，ROM code 中保留了 Patch 函数入口，通过 Patch 可以修改 ROM code 原有行为、扩充 ROM code 功能等。

5. Upper Stack Image

实现了 BLE Upper stack。

6. App Image

开发者开发的 BLE 应用程序，由 Keil 工程编译生成，并经过 fromelf 等工具处理。

9.2 Image 处理工具

Keil 编译直接生成的 application image 是 ELF 格式的，为了便于烧录以及支持 OTA 功能，需要将 ELF 格式的 application image 转换为二进制格式的 bin 文件，并作相应处理。

Demo 工程中使用了 3 个 Command 来处理 application image，分别是 fromelf, prepend_header、md5。

9.2.1 fromelf

Fromelf 来自 Keil（默认安装路径：C:\Keil_v5\ARM\ARMCC\bin\），具体用法可参考 Keil 帮助文档中的 *Using the fromelf Image Converter* 一节。

使用该工具产生烧录的 bin 文件以及反汇编文件。

9.2.2 prepend_header

- 用法：prepend_header.exe [options] ...
- 作用：为支持 OTA 功能，需要在 bin 文件添加校验以及长度信息。

对于 Application Image，其最开始 1024 字节是 image Header，详细请查看 system_rtl876x.c 中定义的 T_IMG_HEADER_FORMAT 结构。

Application Image 的 Checksum 可以是 crc16 或者 sha256，分别记录在 T_IMG_HEADER_FORMAT 结构体中的 crc16 和 sha256 成员，payload 长度记录在 payload_len 成员。

```

1.  typedef struct _IMG_CTRL_HEADER_FORMAT
2.  {
3.      uint8_t ic_type;
4.      uint8_t secure_version;
5.      union
6.      {
7.          uint16_t value;
8.          struct
9.          {
10.             uint16_t xip: 1; // payload is executed on flash
11.             uint16_t enc: 1; // all the payload is encrypted
12.             uint16_t load_when_boot: 1; // load image when boot
13.             uint16_t enc_load: 1; // encrypt load part or not
14.             uint16_t enc_key_select: 3; // referenced to ENC_KEY_SELECT
15.             uint16_t not_ready : 1; //for copy image in ota
16.             uint16_t not_obsolete : 1; //for copy image in ota
17.             uint16_t integrity_check_en_in_boot : 1; // enable image integrity check in boot flow
18.             uint16_t rsvd: 6;
19.         };
20.     } ctrl_flag;
21.     uint16_t image_id;
22.     uint16_t crc16;
23.     uint32_t payload_len;
24. } T_IMG_CTRL_HEADER_FORMAT;

```

```

25.
26. typedef struct _IMG_HEADER_FORMAT
27. {
28.     T_IMG_CTRL_HEADER_FORMAT ctrl_header;
29.     uint8_t uuid[16];
30.     uint32_t exe_base;
31.     uint32_t load_base;
32.     uint32_t load_len;
33.     uint8_t rsvd0[8];
34.     uint32_t magic_pattern;
35.     uint8_t dec_key[16];
36.     uint8_t rsvd1[28];
37.     T_VERSION_FORMAT git_ver;
38.     RSA_PUBLIC_KEY rsaPubKey;
39.     uint8_t sha256[32];
40.     uint8_t rsvd2[76];
41. } T_IMG_HEADER_FORMAT;

```

9.2.3 md5

- 用法: md5.exe [Input Binary Filename]
- 作用: 计算 bin 文件的 MD5, 并且加到 bin 文件中, 用于 MP Tool 烧录时验证。
后缀形式为: [Original File Name]-[MD5].bin

MP Tool 采用 MD5 作为烧录文件的校验值, 并要求将 MD5 值加入到待烧录文件的文件名后缀中。比如原始文件名为 App_ver1.1.0.bin, 经 md5.exe 处理后, 带后缀的文件名为: App_ver1.1.0-d77dd83cb2848d3e9ac04c7dd9367e69.bin。

9.3 烧录方式

Patch Image, Config File 和 App Image 分别支持的下载方式如表 9-1 所示。

表 9-1 Images 可用烧录方式

Tool	Patch Image	Config File	APP Image
Keil	×	×	√
SWDTool	√	√	√
MPTool	√	√	√

10 调试

调试应用程序有以下两种方式。

1. 使用 log 机制跟踪代码的执行和数据。
2. 使用 Keil MDK 和 SWD 进行运行调试，增加/删除断点以及访问/追踪 memory 等。

10.1 Log 机制

Debug analyzer 是用来通过打印 log 的方式来追踪应用程序的执行情况，以及抓取 BT 的数据并进一步通过第三方 BT 分析工具来分析。详情请参考《Debug Analyzer User Manual》。

P0_3 默认作为 Log UART 的输出引脚（也可以将 Log 功能重新配置到其它引脚），经 UART-to-USB 连接到 PC，在 PC 端 COM 口接收 Log 数据。

10.1.1 Debug analyzer

使用步骤如下。

1. 打开 DebugAnalyzer.exe，点击“Settings”，之后点击“Start”即可。

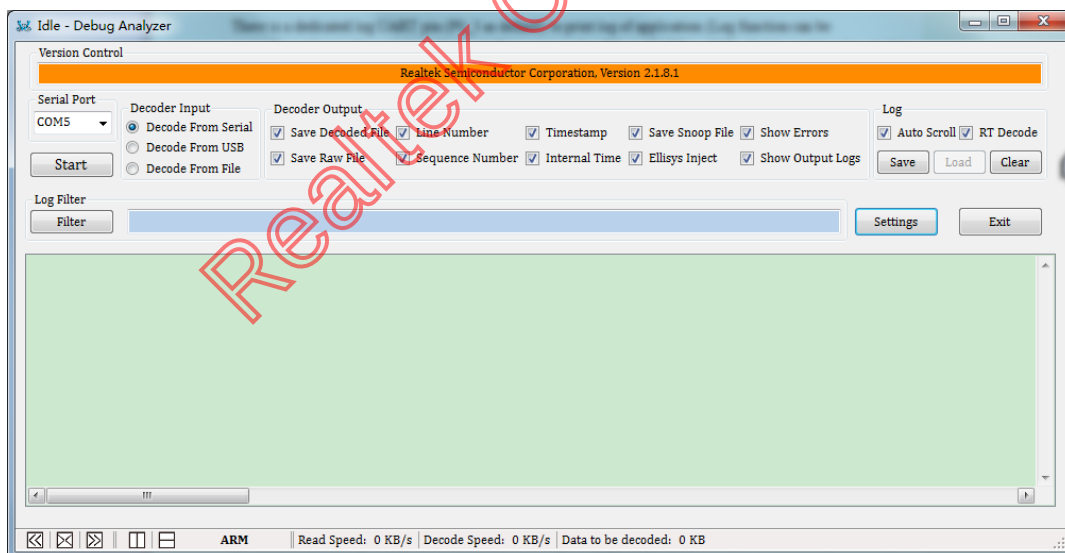


图 10-1 Debug Analyzer 主页面

2. “Settings” 界面中，“Serial Port” 选择 LOG 对应的 COM 口。
3. “App Trace File” 选择对应的 App.trace。

注：App.trace 在 APP 程序编译后和 App.bin 生成在同一个目录位置。

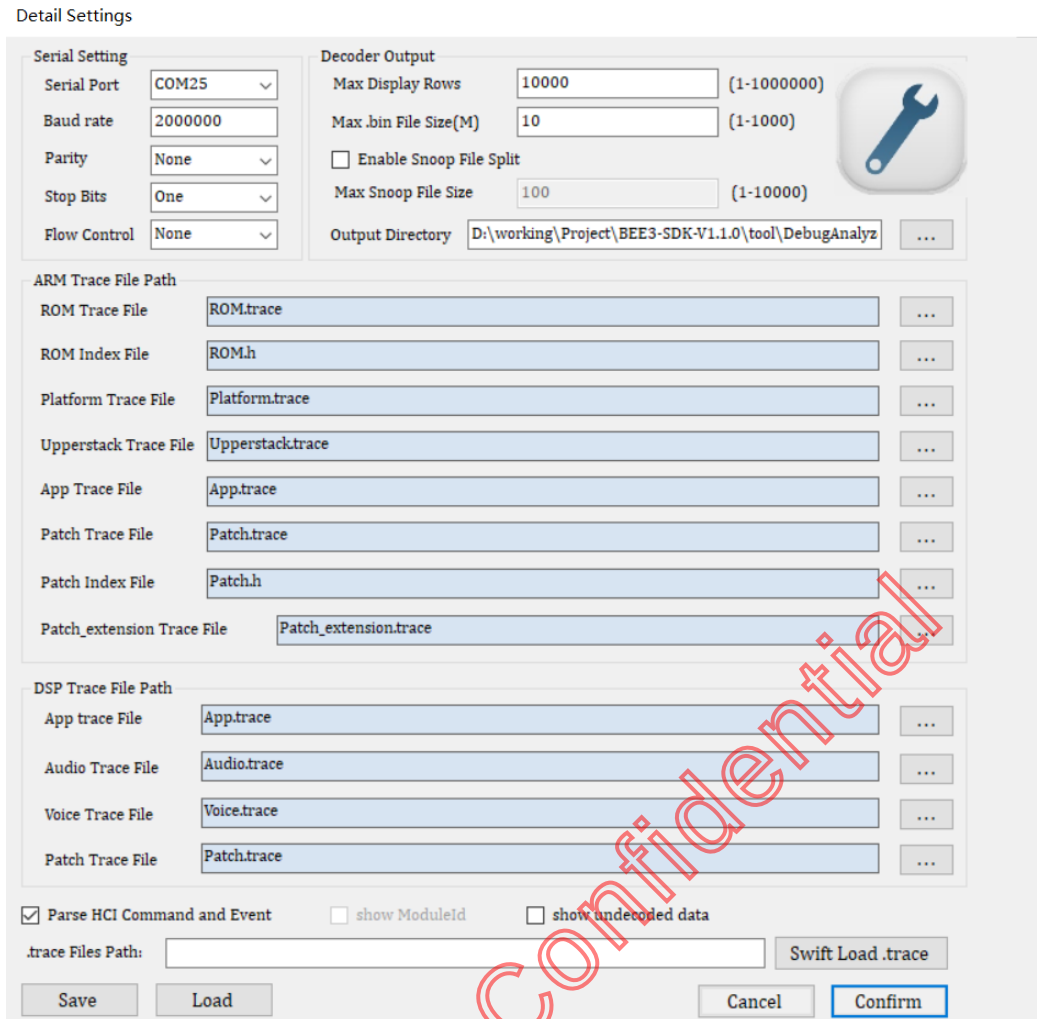


图 10-2 Debug Analyzer 设置页面

Debug analyzer 提供了一些高级功能。

1. **Save Raw File** (建议勾选): 保存 Log 到 xxx.bin 文件, 此文件可提供给 Realtek 做进一步分析。
2. **Log Filter**: 过滤显示, 弹出新窗口, 只显示满足过滤条件的 Log。
3. **keyword search**: 在 Log 打印窗口高亮显示关键字。
4. **Clear Trace**: 清空显示框中的 Log。

注: Debug analyzer 会自动保存 Log 文件, 并按照端口号和生成时间命名, 如 COM 5_2015-06-12_18-05-00.log。

10.1.2 Log 打印基本接口

硬件有特定的 GDMA 通道来打印 Log, log 接口声明如下。

1. `DBG_BUFFER(T_LOG_TYPE type, T_LOG_SUBTYPE sub_type, T_MODULE_ID module, uint8_t level, char* fmt, uint8_t param_num,...)`.

参数 type 固定是 TYPE_BEE3，sub_type 固定是 SUBTYPE_FORMAT。只要关心参数 module 和 level。

T_MODULE_ID 类型中有预定义好几种 module，这些 module 用于将 log 分类别。Debug Analyzer 能够识别这些 module，在每条打印的 log 前自动加到 module 名称。

还有一个概念是 Debug Level，表示 log 要打印的级别，定义了以下 4 种类别。

表 10-1 调试等级

Debug Level	Usage Scenario
LEVEL_ERROR	Fatal, Procedure Cannot Advance (Log Token !!!)
LEVE_WARN	Abnormal Condition Occurred, But Procedure Can Advance (Log Token !!*)
LEVEL_INFO	Important Notification (Log Token !**)
LEVEL_TRACE	Verbose Debug

DBG_BUFFER()接口灵活、功能丰富，但不方便使用。为此，RTL8762E SDK 在 trace.h 中提供了一些易读性更强的 APIs 来包装此接口。

10.1.3 Log 打印封装接口

APIs 通用语法如下。

1. {MODULE}_PRINT_{LEVEL}_{PARAMNUM}(...)

参数释义：

1. MODULE：trace.h 中定义——APP/GAP/USB/FLASH...
2. LEVEL：四种 level——ERROR/WARN/INFO/TRACE
3. PARAMNUM：0 到 8——代表 log 打印参数的个数

- 1) 例如，想要打印一条含 2 个参数的 log。

1. APP_PRINT_WARN2("Test app: ID = %d, data = 0x%x", id, data);

- 2) Debug Analyzer 显示如下这条 Log。

1. 00494 10-13#17:06:45.994 087 02145 [APP] !!*Test app: ID = 3, data = 0xF0

说明：

1. 不要超过 8 个参数（如果使用 DBG_BUFFER()接口，最多允许 20 个参数）
2. 单条 log 中不要超过 128 个字符串
3. 尽量把所有参数放到一条打印语句中

10.1.4 辅助打印接口

DBG_BUFFER()接口只能打印诸如 (d, i, u, o, x) 的整型、字符和指针类型，当要打印 string, binary array 或者 BT 地址时，需用到以下辅助打印接口。

1. TRACE_STRING(char* data)

直接打印字符串，转换符%s。

2. TRACE_BINARY(uint16_t length, uint8_t* data)

以 16 进制格式打印二进制字符串，转换符%b。

3. TRACE_BDADDR(char* bd_addr)

以 BT 地址格式打印二进制字符串，例如：

Hex Array: 0xaa 0xbb 0xcc 0xdd 0xee 0xff → Literal String: FF::EE::DD::CC::BB::AA

说明：单条 Log 中最多能有 4 个地址，转换符%s。

10.1.5 Log 打印示例

下面的例子展示了 log APIs 的一般用法，以及在 Debug Analyzer 上对应的输出。

➤ Log 打印代码

```
1.  uint32_t n = 777777;
2.  uint8_t m = 0x5A;
3.  uint8_t bd1[6] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55};
4.  uint8_t bd2[6] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};
5.  char c1[10] = {'1', '2', '3', '4', '5', '6', '7', '8', '9', '0'};
6.  char c2[8] = {'a', 'b', 'C', 'd', 'E', 'F', 'g', 'H'};
7.  char *s1 = "Hello world!";
8.  char *s2 = "Log Test";
9.  ADC_PRINT_TRACE1("ADC value is %d", n);
10. UART_PRINT_INFO3("Serial data: 0x%x, c1[%b], s1[%s]", m, TRACE_BINARY(10, c1),
    TRACE_STRING(s1));
11. GAP_PRINT_WARN6("n[%d], m[%c] bd1[%s], bd2[%s], c2[%b], s2[%s]", n, m,
    TRACE_BDADDR(bd1), TRACE_BDADDR(bd2),
    TRACE_BINARY(8, c2), TRACE_STRING(s2));
12. APP_PRINT_ERROR0("APP ERROR OCCURED...");
```

➤ Debug Analyzer 上对应的输出结果

1. 00252 10-25#17:12:02.021 132 10241 [ADC] ADC value is 77777
2. 00253 10-25#17:12:02.021 133 10241 [UART] !***Serial data:
0x5a, c1[31-32-33-34-35-36-37-38-39-30], s1[Hello world!]
3. 00254 10-25#17:12:02.022 134 10241 [GAP] !!*n[77777],
m[Z] bd1[55::44::33::22::11::00],
bd2[FF::EE::DD::CC::BB::AA],
c2[61-62-43-64-45-46-67-48], s2[Log Test]
4. 00255 10-25#17:12:02.022 135 10241 [APP] !!!APP ERROR OCCURED...

10.1.6 Log 控制接口

有时候需要控制某些 log 的打开和关闭。一种方式是通过设定 MP Tool 中的 Config File, 将 Config File 烧录到 RTL8762E。如果需要频繁改变 log 打印 level, 这种方式就不够灵活了, 因为每次重烧 Config File 也不方便, 所以 SDK 中提供了 3 个 APIs 来控制特定 module 的特定 level, 分别为:

- log_module_trace_init()
- log_module_trace_set()
- log_module_bitmap_trace_set()

查阅 SDK API 文档获取详细的参数信息。

这里通过一些场景来帮助用户理解如何在应用程序中使用 log 控制的 APIs。假设 log 打印功能已经打开, 并且 Config File 中将所有 log trace mask 置成 1, 意味着所有 module 的所有 level 的 log 都会打印出来。

场景 1: 关闭 APP module 的所有 trace 和 info 级别的 log

```
1. int main(void)
2. {
3.     log_module_trace_set(MODULE_APP, LEVEL_INFO, false);
4.     log_module_trace_set(MODULE_APP, LEVEL_TRACE, false);
5.     ...
6. }
```

场景 2: 只打开 PROFILE module 的 log

```
1. int main(void)
2. {
```

```

3.     uint64_t mask[LEVEL_NUM];
4.     memset(mask, 0, sizeof(mask));
5.     log_module_trace_init(mask);
6.
7.     log_module_trace_set(MODULE_PROFILE, LEVEL_ERROR, true);
8.     log_module_trace_set(MODULE_PROFILE, LEVEL_WARN, true);
9.     log_module_trace_set(MODULE_PROFILE, LEVEL_INFO, true);
10.    log_module_trace_set(MODULE_PROFILE, LEVEL_TRACE, true);
11.    ...
12. }

```

场景 3: 关闭 PROFILE/PROTOCOL/GAP/APP module 的 trace 级别 log

```

1.  int main(void)
2.  {
3.      log_module_bitmap_trace_set(MODULE_BIT_PROFILE | MODULE_BIT_PROTOCOL |
                                   MODULE_BIT_GAP | MODULE_BIT_APP, LEVEL_TRACE, false);
4.      ...
5.  }

```

场景 4: 关闭除了 APP module 以外的所有 trace 和 info 级别的 log, 包括关闭 BT Snoop log

```

1.  int main(void)
2.  {
3.      for (uint8_t i = 0; i < MODULE_NUM; i++)
4.      {
5.          log_module_trace_set((T_MODULE_ID)i, LEVEL_TRACE, false);
6.          log_module_trace_set((T_MODULE_ID)i, LEVEL_INFO, false);
7.      }
8.      log_module_trace_set(MODULE_APP, LEVEL_INFO, true);
9.      log_module_trace_set(MODULE_APP, LEVEL_TRACE, true);
10.     log_module_trace_set(MODULE_SNOOP, LEVEL_ERROR, false);
11.     ...
12. }

```

说明:

如果 MODULE_SNOOP 的 LEVEL_ERROR log 打开, Debug Analyzer 会产生 BT Snoop 的 log 文件 (*.cfa)。如果关闭 MODULE_SNOOP 的 log, 就不会产生 BT Snoop log, 如上场景 2 和场景 4。

10.1.7 DBG_DIRECT

通过调用基于 DBG_BUFFER 接口的函数打印 log 对系统性能的影响比较小,但是打印实时性不高,因为通过这些 APIs 打印的 log 会被缓存到 buffer 中,在系统空闲的时候发送给 Log Uart。

DBG_DIRECT 用来打印实时 log。这个 API 打印 log 对系统的影响比较大,因为会直接发送数据给 Log Uart,在 Log 打印完之前系统不能继续执行,所以强烈建议只在以下一些特定的情况下使用该 API。

1. 在系统重启前
2. 实时性要求高的场景
3. 在 DLPS exit callback 中

➤ 打印 Log 的示例代码

```
1.   DBG_DIRECT("ROM version: %s %s", __DATE__, __TIME__);
```

➤ Debug Analyzer 上对应的输出结果

```
1.   00002 12-22#19:19:32.573 004 00000 ROM version: Dec 22 2017 14:54:04
```

10.2 SWD 调试

如图 10-3 所示, RTL8762E 实现了 3 个基本的调试接口: run control, breakpoint 和 memory access, 以及一个 trace 口: DWT。RTL8762E 支持 SWD, 不支持 JTAG。

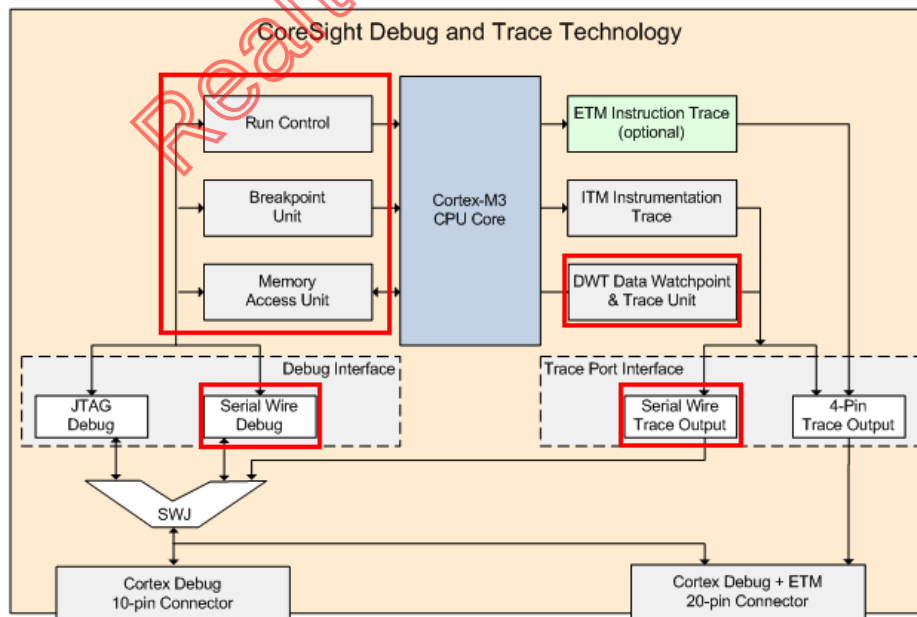


图 10-3 RTL8762E 调试接口

10.2.1 SWD 调试接口

在使用 SWD 之前，需确保已经安装和配置 Keil MDK-ARM 和 SWD debugger。

RTL8762E 支持的主要的调试特性如 UI 上展示。

1. Running control 按钮: Running/Reset/Step
2. Breakpoints 按钮: Add/Delete/Conditional
3. 调试功能窗口:
 - 1) Core registers window: 监视/修改 MCU 寄存器值
 - 2) Disassembly window: 查看反汇编代码，支持断点和 mixed 模式（显示 C 代码）
 - 3) Source code window: 查看 C 代码，支持断点以及变量的实时显示
 - 4) Variable watch window: 查看加入监视的变量值
 - 5) Memory window: 显示/修改待查的内存，支持直接的地址输入和可变的地址输入
 - 6) Call stack and local variable window: 显示当前的调用栈，以及 local 变量（即 stack 上的变量）

注意：有时候可能会遇到 image 无法成功下载到 flash 上，或者即使与 J-link 成功连接，但依旧找不到 SWD，这时可做如下检查：

1. 更改 debug clock 到一个较小的值（例如从 2MHz 降到 1MHz），或者换用更短的 SWD 调试线；
2. IC 可能已经进入了 DLPS 模式，reset IC，这样系统会保持处于 active 状态 5 秒钟。

Reference

- [1]. RTL8762E BLE Stack User Manual.
- [2]. Bee3-SDK.chm
- [3]. RTL8762E Flash User Guide.
- [4]. RTL8762E Memory User Guide
- [5]. RTL8762E Deep Low Power State
- [6]. Debug Analyzer User Manual

Realtek Confidential